

Synapse Reconciliation Engine

Technical Summary & Enterprise-Grade Production Analysis

Repository: `Adrian-Obungu/synapse-reconciliation-engine` **Assessment Date:** June 2026 **Latest Commit:** `fb3749` — Merge PR #6 `feature/fix-grafana-metrics` **Total Commits Reviewed:** 22

Executive Summary

The Synapse Reconciliation Engine has undergone a significant architectural maturation across its most recent commit cycle. What was previously an architectural prototype with mock I/O at every critical path is now a structurally complete, production-aligned middleware service. The core pipeline — from M-Pesa webhook ingestion through idempotency enforcement, schema transformation, persistent storage, and eTIMS compliance submission — is fully implemented and instrumented.

The project is not yet production-deployed, but it has crossed the threshold from prototype to a credible, demonstrable MVP. The remaining gaps are operational rather than architectural: real KRA eTIMS API credentials, a live Daraja sandbox integration test, and a deployment target. The engineering foundation is sound enough to present to technical buyers, payment aggregator partners, or seed-stage investors without qualification.

What Changed in the Latest Commits (PR #5 and PR #6)

The two most recent pull requests represent the most substantive engineering work in the repository's history. PR #5 delivered the full implementation of Options A, B, and C simultaneously. PR #6 fixed the Grafana metrics pipeline and tightened the Prometheus alert evaluation windows for simulation accuracy.

Commit	What It Delivered
1282e30	<code>app/utils/phone.py</code> — pre-compiled regex E.164 normaliser with float/integer coercion
75146d7	<code>app/schemas/etims.py</code> — Pydantic v2 <code>ETIMSInvoicePayload</code> + <code>ETIMSTransformer</code>
b869cf8	<code>app/services/storage.py</code> — <code>StorageEngine</code> facade (Redis idempotency + Postgres ledger)
ea74d59	<code>app/main.py</code> — throttled <code>httpx.AsyncClient</code> on FastAPI lifespan with semaphore
8b6ba04	<code>app/services/etims.py</code> — exponential backoff with jitter and graceful failure
95495f5	Full Docker Compose stack, Postgres DDL, OTel Collector, Prometheus
1060b7b	<code>scripts/simulate.py</code> — 9-scenario deterministic simulation runner
d613385	Grafana dashboard panels, alert rule evaluation window fixes
e5fd9dc	Prometheus metrics endpoints and application dependency wiring

Technical Architecture Assessment

Layer 1 — Ingress & Security Gateway

The FastAPI application correctly implements the Daraja fast-acknowledgement contract. The `/callback` route returns HTTP 200 to Safaricom immediately upon receiving a valid webhook, before any background processing begins. This is not a convenience — it is a hard requirement of the Daraja API specification. Failure to acknowledge within the timeout window causes Daraja to retry the callback, which is why the idempotency guard at this layer is architecturally critical.

The idempotency check uses Redis `SET NX` (set-if-not-exists) with a 24-hour TTL, executed atomically before the background task is dispatched. This means duplicate callbacks — a common occurrence during network instability — are dropped at the gateway layer with a structured `CACHE_HIT` log event, before any database write or outbound API call is made.

The Pydantic v2 schema validation on `MpesaWebhookPayload` enforces structural integrity at the boundary. Malformed payloads are rejected with a 422 before they reach any business logic.

Layer 2 — Phone Normalisation Engine

The `safe_normalize_phone` function in `app/utils/phone.py` is production-quality. It uses a pre-compiled regex (`PHONE_REGEX`) that handles all known Kenyan mobile formats: `07xx` , `011x` , `+254` , bare `254` , and raw 9-digit suffixes. It correctly handles integer inputs (including float representations like `254712345678.0` from JSON numeric fields), strips whitespace and hyphens, and falls back to a configurable sentinel value rather than raising an exception. The pre-compilation is a deliberate performance decision — in an async loop processing high-volume webhook bursts, repeated regex compilation would be measurable overhead.

Layer 3 — eTIMS Schema Transformer

The `ETIMSTransformer` in `app/schemas/etims.py` cleanly separates the Daraja domain from the KRA domain. It extracts `MpesaReceiptNumber` , `Amount` , `PhoneNumber` , and `TransactionDate` from the `CallbackMetadata.Item` array — a nested, dynamically keyed structure that requires careful iteration — and maps them onto a typed `ETIMSInvoicePayload` Pydantic model with `Decimal` precision for the amount field. Using `Decimal` rather than `float` for financial amounts is a correct engineering decision; floating-point arithmetic on currency values introduces rounding errors that compound at scale.

Layer 4 — Dual-Storage State Engine

The `StorageEngine` class manages two connection pools through a single lifecycle facade, initialised in the FastAPI lifespan context and attached to `app.state` . This is the correct pattern for shared async resources in FastAPI — it ensures the pools are created once at startup, shared across all request handlers, and cleanly closed on shutdown.

The Redis layer uses `aioredis` -compatible `redis.asyncio` for the idempotency guard. The Postgres layer uses `asyncpg` connection pooling with a raw parameterised SQL insert. The `io_latency_warning` flag — which fires when a Postgres write exceeds 50ms — is a production telemetry pattern derived from the HDD baseline benchmarking done in earlier commits.

Layer 5 — eTIMS Compliance Service

The outbound HTTP client is instantiated once on the FastAPI lifespan using `httpx.AsyncClient` with `httpx.Limits(max_keepalive_connections=100, max_connections=200)` . An `asyncio.Semaphore` is used to cap concurrent eTIMS submissions, preventing the connection pool from being saturated by burst traffic. The retry loop implements exponential backoff with jitter, which is the correct production pattern — pure exponential backoff without jitter causes retry storms when multiple clients fail simultaneously and retry at the same intervals.

The `MOCK_ETIMS` environment flag allows the outbound KRA call to be bypassed entirely in simulation and CI environments, replaced by a 150ms sleep that mimics realistic network latency.

Layer 6 — Observability Stack

The Docker Compose stack deploys a complete observability pipeline: the OpenTelemetry Collector receives OTLP signals from the application, exports metrics to Prometheus via a Prometheus exporter endpoint, and exports traces to Jaeger via OTLP gRPC. Grafana is provisioned as code with a `datasources.yaml` that registers both Prometheus and Jaeger with stable UUIDs, and a `dashboards.yaml` with `disableDeletion: true` enforcing configuration-as-code discipline.

The Grafana `Synapse Live Performance` dashboard provides three live panels: application uptime status, HTTP request volume rate, and a security anomalies detector. The Prometheus alert rules cover four conditions: `RetryExhaustion` (any failed eTIMS submission), `AuthenticationFailures` (brute force signature — >3 401s in 1 minute), `HighLatencyAnomaly` (average request latency >500ms), and `AppDown` (service unreachable for >10 seconds).

Test Coverage Assessment

The test suite covers the three most critical units of business logic.

Test File	What It Covers	Quality
<code>test_phone.py</code>	14 assertions across valid formats, integer/float inputs, formatting edge cases, invalid inputs, and custom fallback	Strong — covers all realistic dirty data patterns
<code>test_etims_schema.py</code>	Full transformer pipeline with a realistic Daraja payload, asserting field mapping, type correctness, and Decimal precision	Strong — tests the exact production code path
<code>test_logging.py</code>	Structured log output validation	Adequate
<code>test_main.py</code>	FastAPI app startup and health check	Adequate

The test suite does not yet cover the `StorageEngine` (Redis and Postgres integration tests), the retry loop behaviour in `etims.py` , or the idempotency guard under concurrent load. These are the gaps to close before claiming production readiness.

Enterprise-Grade Production Readiness Gap Analysis

The following table provides an honest assessment of each dimension against enterprise production standards.

Dimension	Current State	Enterprise Standard	Gap
Schema validation	Pydantic v2, strict typing, Decimal for currency	Full boundary validation with error codes	Minor — error response schema not standardised
Idempotency	Redis <code>SET NX</code> with 24h TTL	Atomic, durable, distributed	Solid — meets the standard
Storage	<code>asyncpg</code> pool, parameterised SQL, latency telemetry	Connection pooling, retry on transient failure, migrations	Storage retry on transient DB failure not implemented
HTTP client	Shared pool, semaphore, backoff with jitter	mTLS, circuit breaker, dead letter queue	No circuit breaker; no DLQ for permanently failed invoices
Authentication	None	API key or OAuth2 per merchant	Missing entirely — critical for multi-tenant use
Database migrations	Raw <code>init.sql</code>	Alembic or Flyway versioned migrations	No migration tooling
Secrets management	<code>.env</code> file via pydantic-settings	Vault, AWS Secrets Manager, or Kubernetes Secrets	Adequate for PoC, not for production
Observability	OTel Collector + Prometheus + Jaeger + Grafana	Full tracing, structured logs, alerting	Solid foundation — spans not yet correlated to business events
Test coverage	Unit tests on phone, schema, main	Integration tests, contract tests, load tests	Storage and retry path tests missing

Dimension	Current State	Enterprise Standard	Gap
CI/CD	None	GitHub Actions with test, lint, build, deploy	Not present
Deployment	Docker Compose	Kubernetes or managed container service	Compose is adequate for PoC; not for production scale

Scaling Assessment

The architecture is horizontally scalable by design. The FastAPI application is stateless — all shared state lives in Redis and Postgres, not in application memory. Multiple instances of the app container can run behind a load balancer without coordination, because the Redis idempotency guard is atomic across all instances. The `asyncpg` connection pool is per-instance, so scaling to N instances multiplies the effective pool size by N.

The current Postgres schema has appropriate indexes on `checkout_request_id` (unique), `mpesa_receipt_number`, and `created_at`. At Kenyan SME transaction volumes — typically hundreds to low thousands of transactions per day per merchant — a single Postgres instance handles this comfortably. At aggregator scale (millions of transactions per day across thousands of merchants), read replicas and table partitioning by `created_at` would be the next steps.

The OTel Collector is the observability bottleneck at scale. The current configuration uses in-memory batching. At high volume, the Collector should be deployed as a DaemonSet (Kubernetes) or scaled independently with persistent queue storage.

Strategic Position Summary

The project is architected for the right problem at the right time. The KRA eTIMS mandate creates a compliance obligation that no VAT-registered M-Pesa merchant can ignore, and the engineering work done in the latest commits demonstrates a clear understanding of both the Daraja and eTIMS API contracts. The gap between the current state and a production-deployable, revenue-generating service is well-defined and achievable.

The three most impactful next steps, in order of priority, are: implementing merchant authentication (without it, the service cannot be multi-tenant), adding a dead letter queue for permanently failed eTIMS submissions (without it, failed invoices are silently lost), and obtaining

KRA eTIMS sandbox credentials to validate the outbound API call against the real compliance endpoint.

Synapse Reconciliation Engine — Enterprise Assessment Prepared June 2026 | Based on commit `fb3749`